



PROJECT-BASED LEARNING

PHASE-I EVALUATION

Project Proposal: Urban Emergency Pre-emption Engine (UEPE)

Team ID: DSCPP-III-2026-T319

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details



Team Information

Team ID: *DSCPP-III-2026-T319*

Team Name: *Avengers*

Domain: *Urban Emergency Vehicle Signal Pre-emption*

Team Members

1. Mohit Singh Basera (Team Lead) – 2027920
2. Pranav Pandey – 2027984
3. Riddhi – 2028472

Mentor

Ram Ji Chauchan

Interactions completed:



Problem Statement

Emergency vehicles lose critical time at intersections that have no way of knowing an emergency is in progress. Existing pre-emption systems (RF emitters, GPS-linked overrides) are expensive, closed-source, and give no visibility into how the decision to clear a signal is made.

Motivation

- ▶ Make the routing and signal-clearance logic fully visible and traceable.
- ▶ Handle multiple emergency vehicles converging on the same intersection.
- ▶ Provide a transparent alternative to closed-source commercial systems.

Target Users / Stakeholders

- ▶ City traffic management authorities.
- ▶ Emergency dispatch / CAD systems.
- ▶ Future integration with real traffic-signal hardware.



Project Objectives

1. Design an emergency-vehicle signal pre-emption simulator using custom data structures in C/C++.
2. Represent the city road network as a graph (adjacency list) of intersections and road segments.
3. Implement Dijkstra's / A* shortest-path algorithm to compute the fastest route and ETA.
4. Use a priority queue to arbitrate simultaneous pre-emption requests at an intersection.
5. Use a hash table for $O(1)$ intersection lookup and live signal state.
6. Generate reports on pre-emption delay saved and busiest corridors using sorting algorithms.

Key Objective:

Show how graph, priority-queue and hash-table choices affect pre-emption latency and correctness.



Proposed Approach

1. Model the road network as an adjacency-list graph.
2. Compute the fastest route and ETA using Dijkstra's / A*.
3. Queue competing pre-emption requests by vehicle priority and arrival time.
4. Signal controller pre-empts the normal cycle and resolves conflicts.
5. Restore normal cycling after clearance; log every override.

Key Features

- ▶ Graph-based road network model
- ▶ Priority-based request arbitration
- ▶ Hash-indexed intersection registry
- ▶ Dijkstra's / A* routing engine
- ▶ Sorting-based delay & corridor reports

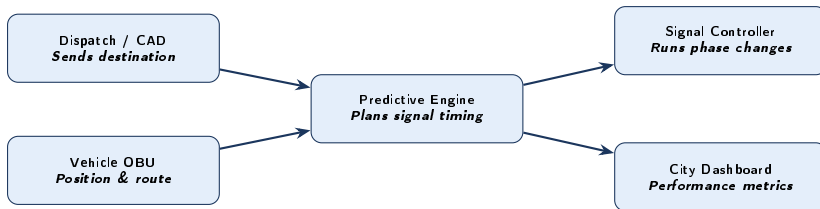
Integration of PBL Course Concepts



Course	Concepts Applied	Project Module
Data Structures in C	<i>Graph (adjacency list), priority queue, hash table, FIFO queue</i>	<i>Road network, pre-emption queue, intersection registry</i>
OOPs with C++	<i>Dijkstra's / A* algorithm; class hierarchy (Intersection, EmergencyVehicle, RouteRequest, SignalController, PreemptionManager)</i>	<i>Routing engine and signal control</i>
Algorithms / DBMS	<i>Sorting algorithms (quicksort/mergesort); CSV-based simulated data handling</i>	<i>Reporting module, core implementation</i>

Requirement:

the project combines data structures, algorithms and C++ OOP design in one pre-emption simulation.



Major Components

- ▶ Road-network graph
- ▶ Routing module (Dijkstra's / A*)
- ▶ Pre-emption priority queue
- ▶ Hash-indexed intersection registry
- ▶ Signal controller

Data / Control Flow

- ▶ Vehicle OBU sends position & route to the predictive engine.
- ▶ Dispatch / CAD sends the destination for a new emergency call.
- ▶ Predictive engine plans signal timing and forwards it to the signal controller.
- ▶ City dashboard shows live performance metrics.



Technology Stack

- ▶ Programming: C (data structures) + C++ (OOP interface)
- ▶ Data structures: graph, priority queue, hash table, FIFO queue
- ▶ Algorithms: Dijkstra's / A* shortest path; quicksort / mergesort
- ▶ Environment: VS Code / Code::Blocks, GCC / G++
- ▶ Version Control: Git / GitHub

Development Methodology

1. Requirement and problem analysis
2. Core network model design (Milestone 1)
3. Routing engine implementation (Milestone 2)
4. Pre-emption queue & registry (Milestone 3)
5. Signal state & conflict resolution (Milestone 4)
6. Reporting & final integration (Milestone 5)



Progress Achieved

- ▶ Project proposal, motivation and state-of-the-art review completed.
- ▶ Goals and milestones defined.
- ▶ Core data structures (graph, heap, hash table) and Dijkstra's / A* algorithm selected.

Individual Contribution

Member		Role	Contribution
Mohit Basera	Singh	Team Lead	Team leadership and project coordination
Pranav Pandey		Team Member	—
Riddhi		Team Member	Report formation and data collection

Mentor: *Ram Ji Chauchan*



Roadmap

1. Milestone 1: Core Network Model
2. Milestone 2: Routing Engine
3. Milestone 3: Pre-emption Request Queue
4. Milestone 4: Signal State & Conflict Resolution
5. Milestone 5: Reporting & Final Integration

Expected Outcomes

- ▶ Fully functional, console-based UEPE application in C/C++.
- ▶ Graph-based routing with Dijkstra's / A*.
- ▶ Priority-based pre-emption arbitration.
- ▶ Activity reports on delay saved and busiest corridors.

References

1. GeeksforGeeks — data structure implementation guides
2. Deitel & Deitel, *C++ How to Program*
3. Global Traffic Technologies (Opticom) — emergency-vehicle pre-emption architecture reference
4. Cormen, Leiserson, Rivest, Stein — *Introduction to Algorithms*



Thank You

Questions & Suggestions